

Universidad ORT Uruguay
Facultad de Ingenieria

Automatizacion Git y Sistema de Ventas

Taller de Tecnologias 1

Fabrizio Pedemonti - N 372959

Lucas Pittaluga - N 372926

Agustin Roizen - N 350021

Docente: Natalia Jauregui

2026

Declaracion de Autoria

Nosotros, Fabrizio Pedemonti, Lucas Pittaluga y Agustin Roizen, declaramos que el trabajo que se presenta en esta obra es de nuestra propia mano. Podemos asegurar que:

- La obra fue producida en su totalidad mientras realizabamos Taller de Tecnologias 1;
- Cuando hemos consultado el trabajo publicado por otros, lo hemos atribuido con claridad;
- Cuando hemos citado obras de otros, hemos indicado las fuentes. Con excepcion de estas citas, la obra es enteramente nuestra;
- En la obra, hemos acusado recibo de las ayudas recibidas;
- Cuando la obra se basa en trabajo realizado conjuntamente con otros, hemos explicado claramente que fue contribuido por otros, y que fue contribuido por nosotros;
- Ninguna parte de este trabajo ha sido publicada previamente a su entrega, excepto donde se han realizado las aclaraciones correspondientes.

Abstract

Este trabajo cubre dos partes: un sistema de ventas en Bash y un script que sube cambios a GitHub de forma automatica cada semana. El sistema de ventas deja registrar usuarios, iniciar sesion, dar de alta productos y comprarlos, todo desde la terminal. Los datos vuelcan a un archivo de texto plano. El script de automatizacion (weekly_push.sh) revisa si hay cambios locales, los commitea y los pushea, y si algo falla lo registra en el README. Tambien configuramos una tarea cron para que corra los domingos a las 9. Del lado de gestion, usamos Jira con Kanban para dividir el trabajo en 19 tareas, y creamos templates de issues en GitHub para bugs, features, mejoras y documentacion. Todo el desarrollo siguio GitFlow con ramas feature.

Indice

1 Introduccion	5
2 Organizacion del Trabajo	6
3 Inicializacion del repositorio y GitFlow	7
3.1 Estructura de ramas	7
3.2 Flujo de merges	7
4 Sistema de Ventas (ventas.sh)	8
4.1 Menu interactivo	8
4.2 Registro de usuarios	8
4.3 Inicio y cierre de sesion	8
4.4 Alta de productos	8
4.5 Compra de productos	9
5 Script de automatizacion semanal (weekly_push.sh)	9
5.1 Funcionamiento	9
5.2 Detalle tecnico	9
6 Configuracion de Cron	10
7 GitHub Issues Templates	11
8 Conclusiones	12
Bibliografia	13

1. Introduccion

El objetivo del proyecto era doble. Por un lado, escribir un script Bash que subiera cambios a GitHub de forma automatica todas las semanas. Por el otro, implementar un sistema de ventas desde la terminal, con usuarios, productos y compras. Todo organizado con GitFlow y ramas feature.

El equipo se dividio el trabajo en 19 tareas repartidas en tres areas: sistema de ventas, automatizacion Git, y documentacion con templates de issues.

2. Organizacion del Trabajo

Usamos Jira en modo Kanban para llevar el día a día de las tareas.

- Sistema de ventas: menu, registro de usuarios, login, alta de productos, compras.
- Automatizacion Git: weekly_push.sh y configuracion de cron.
- Documentacion: templates de issues en .github/ISSUE_TEMPLATE/ y ejemplos.

Cada funcionalidad tuvo su propia rama feature. Se mergeaba a develop cuando estaba lista, y develop se fusionaba a main al alcanzar un estado estable.

3. Inicializacion del repositorio y GitFlow

3.1. Estructura de ramas

El repo arranco con git init en GitHub. Las ramas que usamos:

Rama	Proposito
main	Produccion / Version estable
develop	Integracion de features
feature/autenticacion	Registro, login y logout
feature/alta-productos	Alta de productos
feature/venta-productos	Compra de productos

3.2. Flujo de merges

Cada feature se desarrollo en su rama y se integro a develop. Cuando develop estaba en un punto estable, se mergeaba a main. La Figura 1 muestra el arbol resultante.

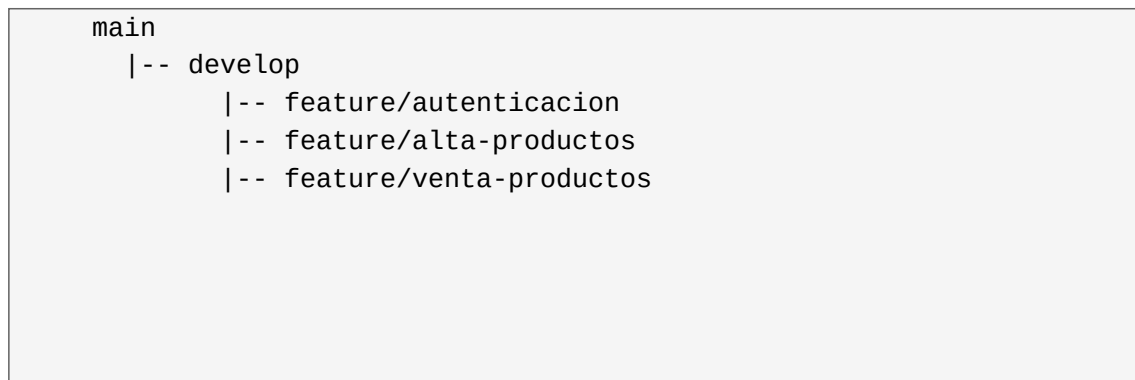


Figura 1. Estructura de ramas del repositorio

4. Sistema de Ventas (ventas.sh)

El sistema de ventas es un solo script Bash con un menu que va mostrando opciones segun si el usuario esta logueado o no.

4.1. Menu interactivo

Un bucle while true muestra las opciones, un case las maneja, y read captura lo que el usuario elige. El estado de la sesion se ve siempre en pantalla.

4.2. Registro de usuarios

registrar_usuario() pide nombre y contrasena. Rechaza si estan vacios o tienen espacios. Usa grep -q para ver si el usuario ya existe, y si no, lo guarda como USUARIO:nombre clave en data/ventas.dat.

4.3. Inicio y cierre de sesion

iniciar_sesion() revisa las credenciales contra el archivo y deja la variable usuario_logueado con el nombre del usuario. cerrar_sesion() la limpia.

4.4. Alta de productos

alta_producto() solo funciona si hay sesion activa. Pide nombre, descripcion, precio y stock. Valida que no esten vacios y que precio/stock sean numeros enteros.

4.5. Compra de productos

comprar_producto() busca el producto con grep, fija si hay stock, y si hay actualiza el archivo: saca la linea vieja con grep -v, escribe la nueva en un archivo temporal, y lo renombra con mv.

5. Script de automatizacion semanal (weekly_push.sh)

El script vive en scripts/weekly_push.sh. Se encarga de sincronizar el repositorio local con GitHub sin que nadie tenga que acordarse.

5.1. Funcionamiento

En criollo, hace lo siguiente:

- 1. Baja los ultimos cambios con git pull.
- 2. Ve si hay cambios locales con git status --porcelain.
- 3. Si no hay: anota una alerta en README.md, commitea y pushea igual.
- 4. Si hay: agrega todo con git add -A, excluye README.md con git reset, commitea y pushea.
- 5. Si el push sale bien: escribe "exito" y las lineas tocadas en README.md.
- 6. Si falla: escribe el error en README.md.

5.2. Detalle tecnico

Un problema que nos encontramos: si README.md se modificaba antes del push, el registro de "exito" se escribia aunque el push despues fallara. La solucion fue separar el README del commit con git add -A && git reset README.md. Asi el registro se escribe solo despues de confirmar que el push funciona.

6. Configuración de Cron

Agregamos una entrada en crontab para que el script corra solo los domingos a las 9 de la mañana:

```
0 9 * * 0 /ruta/completa/obligatorio_taller/scripts/weekly_push.sh
```

El orden es el clásico: minuto, hora, día del mes, mes, día de la semana (0 es domingo).

7. GitHub Issues Templates

Armamos 4 templates en `.github/ISSUE_TEMPLATE/` y abrimos 8 issues de ejemplo (dos por cada tipo). La tabla muestra que cubre cada uno:

Archivo	Tipo	Campos
bug.yml	Reporte de error	Descripcion, pasos para reproducir, comportamiento
feature_request.yml	Solicitud de funcionalidad	Problema relacionado, solucion propuesta, alternativas
mejoras.yml	Propuesta de mejora	Area de mejora, beneficio esperado, ejemplo de uso
documentacion.yml	Documentacion	Seccion afectada, que falta aclarar, propuesta de contenido

Los issues estan en github.com/bocho8/obligatorio_taller/issues.

8. Conclusiones

Cerrando, esto es lo que quedo funcionando:

- Un sistema de ventas en Bash con usuarios, productos y compras.
- Un script que sube cambios a GitHub solo, sin que nadie se acuerde.
- Cron configurado para que el script corra los domingos.
- Templates de issues para mantener el repo ordenado.
- GitFlow con ramas feature y merges estructurados.

Jira con Kanban nos ayudo a organizar las 19 tareas y a repartir el trabajo de forma mas o menos pareja entre los tres.

Bibliografia

- [1] Pro Git Book, S. Chacon and B. Straub, "Git Branching - Branching Workflows," disponible en: <https://git-scm.com/book/en/v2/Git-Branching-Branching-Workflows>. [Accedido: junio 2025].
- [2] GNU Bash Manual, "Bash Reference Manual," Free Software Foundation, disponible en: <https://www.gnu.org/software/bash/manual/>. [Accedido: junio 2025].
- [3] Atlassian, "Jira Software - Kanban," disponible en: <https://www.atlassian.com/software/jira/guides/kanban>. [Accedido: junio 2025].
- [4] GitHub Docs, "About issue templates," disponible en: <https://docs.github.com/en/communities/using-templates-to-encourage-useful-issues-and-pull-requests/about-issue-templates>. [Accedido: junio 2025].
- [5] ORT Universidad, "Normas especificas para la presentacion de trabajos finales de carrera (TFDC) - Documento 302-ET," Facultad de Ingenieria - Escuela de Tecnologia, 2020.